

PokéSector 35

(Memoria técnica)

Aplicación Fullstack de Juego Retro
React + Express + PostgreSQL
Mayo 2026

1. RESUMEN EJECUTIVO

PokéSector 35 es una aplicación web fullstack que recrea la experiencia de los juegos clásicos de Pokémon en un formato de navegador moderno.

El proyecto combina tecnologías actuales (React, Express, PostgreSQL) con una estética retro inspirada en Game Boy.

Características principales:

- Sistema de exploración en mapa 5×7 celdas
- Mecánicas de captura de Pokémon de la 1ª generación (151 especies)
- Sistema de autenticación JWT con refresh tokens
- Ranking global por dificultad
- Persistencia de partidas con hasta 3 slots por usuario
- Sistema de replay para reproducir partidas
- Panel administrativo con gestión completa
- API REST documentada con Swagger OpenAPI 3.0

2. OBJETIVOS DEL PROYECTO

2.1 Objetivo General

Desarrollar una aplicación web fullstack que permita a los usuarios jugar una versión simplificada de Pokémon en el navegador, con persistencia de datos, autenticación segura y un panel administrativo completo.

2.2 Objetivos Específicos

- Implementar un sistema de juego funcional con mecánicas de exploración y captura
- Desarrollar una API REST completa y documentada
- Garantizar la seguridad mediante autenticación JWT y validaciones
- Crear un frontend responsive con estética retro
- Implementar persistencia de datos en PostgreSQL
- Desarrollar un panel administrativo para gestión
- Documentar el proyecto de forma profesional

3. ARQUITECTURA DEL SISTEMA

3.1 Visión General

El sistema sigue una arquitectura de tres capas:

Capa	Tecnología	Responsabilidad
Frontend	React 18.2 + Vite	Interfaz de usuario, lógica de juego cliente
Backend	Express.js + Node.js	API REST, autenticación, lógica de negocio
Base de Datos	PostgreSQL 15	Persistencia de usuarios, partidas, rankings

3.2 Stack Tecnológico

Backend:

- Node.js 18+
- Express.js 4.18
- Sequelize ORM 6.35
- PostgreSQL 15
- JWT (jsonwebtoken) para autenticación
- Bcrypt para hash de contraseñas
- Swagger (swagger-ui-express) para documentación

Frontend:

- React 18.2
- Vite 6.0
- JavaScript ES6+
- CSS3

Herramientas:

- Docker & Docker Compose
- Git para control de versiones
- npm para gestión de paquetes

4. BASE DE DATOS

4.1 Esquema de Tablas

Tabla	Descripción	Campos Clave
users	Usuarios registrados	id, username, password_hash, role
game_slots	Partidas guardadas	id, user_id, slot_number, explorer, difficulty
captured_pokemon	Pokédex de usuarios	id, slot_id, pokemon_id, pokemon_name
game_replays	Historial de movimientos	id, slot_id, moves (JSON)
rankings	Posiciones globales	id, user_id, difficulty, best_score
refresh_tokens	Tokens de sesión	id, user_id, token, expires_at

4.2 Relaciones

- users (1) → game_slots (N)
- users (1) → refresh_tokens (N)
- users (1) → rankings (N)
- game_slots (1) → captured_pokemon (N)
- game_slots (1) → game_replays (1)

5. API REST

5.1 Endpoints Principales

Método	Endpoint	Descripción
POST	/api/auth/register	Registra nuevo usuario
POST	/api/auth/login	Autentica usuario (admin)
POST	/api/auth/login-game	Login desde juego
POST	/api/auth/refresh	Renueva access token
GET	/api/users	Lista usuarios
GET	/api/users/:id	Obtiene usuario
POST	/api/users/:userId/slots	Crea partida
GET	/api/ranking	Obtiene ranking global

Total: ~25 endpoints documentados en Swagger

5.2 Documentación Swagger

La API está completamente documentada con Swagger OpenAPI 3.0, accesible en: <http://localhost:3000/api-docs>

Incluye:

- Esquemas de request/response
- Ejemplos de uso
- Códigos de error
- Pruebas interactivas

6. SEGURIDAD

6.1 Autenticación JWT

- Access Token: válido 1 hora
- Refresh Token: válido 7 días, almacenado en BD
- Tokens firmados con HS256
- Middleware de validación en rutas protegidas

6.2 Protección de Contraseñas

- Hash con bcrypt (10 rounds)
- No se almacenan contraseñas en texto plano
- Validación de complejidad mínima (6 caracteres)

6.3 Rate Limiting

- Login: máx 5 intentos cada 15 segundos
- Registro: máx 5 intentos cada 15 segundos
- Prevención de ataques de fuerza bruta

6.4 Validaciones

- Anti SQL Injection en usernames
- Anti XSS en entradas de usuario
- Validación de URLs en inputs
- Sanitización de datos

7. FRONTEND

7.1 Componentes Principales

- GameScreen: pantalla principal del juego
- BattleScreen: sistema de batalla contra Pokémon
- MenuScreen: navegación de menús
- StatsScreen: estadísticas del jugador
- GameControls: controles táctiles/teclado

7.2 Lógica de Juego

- Mapa de exploración: 5 columnas × 7 filas (35 celdas)
- Movimiento del jugador en 4 direcciones
- Encuentros aleatorios con Pokémon (probabilidad variable)
- Sistema de captura con pokéballs

- HP del jugador (10 puntos máximo)
- 4 dificultades: Fácil, Normal, Difícil, Infernal

7.3 Persistencia

- localStorage: para usuarios no registrados
- API REST: para usuarios registrados
- Sincronización automática al registrarse

8. FUNCIONALIDADES PRINCIPALES

8.1 Juego

- Exploración de mapa 5×7
- Encuentros aleatorios con Pokémon
- Sistema de captura
- Pokédex con 151 Pokémon
- Sistema de HP
- 4 niveles de dificultad
- Control por teclado, gamepad y ratón/táctil

8.2 Usuario

- Registro y login
- Hasta 3 partidas guardadas simultáneas
- Pokédex personal
- Estadísticas de juego
- Historial de movimientos (replay)

8.3 Ranking

- Ranking global por dificultad
- Mejor puntuación registrada
- Total de partidas completadas
- Actualización automática

8.4 Administración

- Panel web completo
- Gestión de usuarios (CRUD)
- Visualización de partidas
- Gestión de rankings
- Estadísticas globales

9. INSTALACIÓN Y DESPLIEGUE

9.1 Requisitos del Sistema

- Node.js 18 o superior
- PostgreSQL 15 o superior
- npm 9 o superior
- 2GB RAM mínimo
- 500MB espacio en disco

9.2 Instalación Backend

- Crear base de datos PostgreSQL
- Configurar variables de entorno (.env)
- Instalar dependencias: `npm install`
- Ejecutar SQL de creación de tablas
- Iniciar servidor: `npm run dev`

9.3 Instalación Frontend

- Instalar dependencias: `npm install`
- Configurar URL del backend (.env)
- Iniciar en desarrollo: `npm run dev`
- Compilar para producción: `npm run build`

10. DOCUMENTACIÓN

El proyecto incluye documentación completa en la carpeta /docs:

- INDEX.md - Índice y navegación
- QUICKSTART.md - Guía rápida (5 minutos)
- SETUP.md - Instalación detallada
- ARQUITECTURA.md - Visión general del sistema
- BACKEND.md - Estructura del servidor
- API.md - Referencia completa de endpoints
- MODELOS.md - Esquema de base de datos
- AUTENTICACION.md - Sistema de seguridad
- FRONTEND.md - Componentes React y lógica de juego

11. CONCLUSIONES

PokéSector 35 demuestra la capacidad de integrar tecnologías web modernas para crear experiencias de juego inmersivas en el navegador. El proyecto cumple con los siguientes objetivos técnicos:

- Arquitectura fullstack bien estructurada
- API REST completa y documentada
- Sistema de autenticación robusto
- Persistencia de datos eficiente
- Frontend responsive con buena UX
- Código limpio y mantenible
- Documentación profesional

El resultado es una aplicación funcional, escalable y lista para producción que puede servir como base para futuros desarrollos o como referencia educativa para proyectos fullstack.

12. APÉNDICES

ATENCIÓN: Los valores y claves que se dan a continuación son para poder levantar el proyecto en local, probar el juego, su base de datos y testear todo el conjunto sin problemas.

En ningún caso estos valores representarán los valores reales del juego una vez esté finalizado.

A. Variables de Entorno

Backend (.env):

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=pokesector_db
DB_USER=pokesector_user
DB_PASSWORD=tu_contraseña
JWT_SECRET=tu_secreto_jwt
REFRESH_SECRET=tu_secreto_refresh
PORT=3000
```

Frontend (.env):

```
VITE_API_URL=http://localhost:3000
```

B. Endpoints Completos

Autenticación: 5 endpoints

Usuarios: 8 endpoints

Slots: 5 endpoints

Pokédex: 2 endpoints

Ranking: 2 endpoints

Replay: 2 endpoints

Total: ~25 endpoints

C. Tecnologías y Versiones

Backend:

- express: 4.18.2
- sequelize: 6.35.2
- jsonwebtoken: 9.0.2
- bcrypt: 5.1.1
- swagger-ui-express: 5.0.0

Frontend:

- react: 18.2.0
- vite: 6.0.0